

User Manual

for S32 SERDES Driver

Document Number: UM11SERDESASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	7
3.7 Runtime errors	8
3.8 Symbolic Names Disclaimer	9
4 Tresos Configuration Plug-in	10
4.1 Module Serdes	11
4.2 Container SerdesGeneral	11
4.3 Parameter SerdesDevErrorDetect	13
4.4 Parameter SerdesSetModeApi	13
4.5 Parameter SerdesVersionInfoApi	14
4.6 Parameter SerdesEnableUserModeSupport	14
4.7 Parameter SerdesMulticoreSupport	15
4.8 Parameter SerdesTimeoutMethod	15
4.9 Parameter SerdesTimeoutDuration	16
4.10 Parameter SerdesMainFunctionPeriod	16
4.11 Parameter SerdesJobDone	16
4.12 Parameter SerdesJobFailed	17
4.13 Reference SerdesEcucPartitionRef	17
4.14 Container SerdesGlobalConfig	18
4.15 Container SerdesChannel	18
4.16 Parameter SerdesChannelId	19
4.17 Parameter SerdesHwChannel	19
4.18 Parameter SerdesMode	20
4.19 Parameter SerdesPhyClock	20
4.20 Parameter SerdesClock	21
4.21 Parameter UseXpcs0	21

4.22	Parameter UseXpcs1	22
4.23	Reference SerdesChannelEcucPartitionRef	22
4.24	Container XPCS_0	22
4.25	Parameter XpcsSpeed0	23
4.26	Parameter XpcsDuplex0	23
4.27	Parameter XPCS0_AN	24
4.28	Parameter SerdesLoopback_0	24
4.29	Container XPCS_1	25
4.30	Parameter XpcsSpeed1	25
4.31	Parameter XpcsDuplex1	26
4.32	Parameter XPCS1_AN	26
4.33	Parameter SerdesLoopback_1	27
4.34	Container CommonPublishedInformation	27
4.35	Parameter ArReleaseMajorVersion	28
4.36	Parameter ArReleaseMinorVersion	28
4.37	Parameter ArReleaseRevisionVersion	29
4.38	Parameter ModuleId	29
4.39	Parameter SwMajorVersion	30
4.40	Parameter SwMinorVersion	30
4.41	Parameter SwPatchVersion	31
4.42	Parameter VendorApiInfix	31
4.43	Parameter VendorId	32
5	Module Index	33
5.1	Software Specification	33
6	Module Documentation	34
6.1	Serdes HLD	34
6.1.1	Detailed Description	34
6.1.2	Data Structure Documentation	35
6.1.3	Macro Definition Documentation	36
6.1.4	Types Reference	37
6.1.5	Enum Reference	37
6.1.6	Function Reference	38
6.2	Serdes IPL	40
6.2.1	Detailed Description	40
6.2.2	Data Structure Documentation	40
6.2.3	Function Reference	43



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor Serdes for S32. Serdes driver configuration parameters and deviations from the specification are described in Driver chapter of this document. Serdes driver requirements and APIs are described in the Serdes driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
ECU	Electronic Control Unit
ISR	Interrupt Service Routine
N/A	Not Applicable
VLE	Variable Length Encoding

2.5 Reference List

#	Title	Version
1	Reference Manual	S32G2 Reference Manual, Rev. 7, February 2023
		S32G3 Reference Manual, Rev.3, Feb 2023
2	Datasheet	S32G2 Data Sheet, Rev 6, Dec 2022
		S32G3 Data Sheet, Rev 2, RC, Feb 2023
		VR5510 Data Sheet, Rev 5, April 2022
3	Errata	S32G2: Mask Set Errata for Mask 0P77B v3.0
		S32G2: Mask Set Errata for Mask 1P77B v1.0
		S32G3: Mask Set Errata for Mask 1P72B v2.1

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

N/A

3.2 Driver Design Summary

The Serdes driver is implemented as an Autosar complex device driver. It uses the Serdes hardware peripheral which provides support for SGMII and PCIe. Serdes driver supports several operating mode such as: both lanes PCIe, both lanes SGMII_X2 or combo mode (one PHY lane PCIe and the other SGMII). Hardware and software settings can be configured using an Autosar standard configuration tool.

Serdes initialization is asynchronous. [Serdes_Init\(\)](#) function will only start the initialization, to complete it, [Serdes_↵_MainFunction\(\)](#) should be called multiple times, until the module is fully initialized. To check if the initialization job is completed, [Serdes_GetJobResult\(\)](#) should be called. If returned value is SERDES_INIT_OK, initialization is finished.

Function [Serdes_GetStatus\(\)](#) can be used to check the initialization of each serdes channel. If returned value is SERDES_SUCCESS, the initialization for the specified serdes channel is completed.

Serdes functionalities:

- initialization of Serdes ([Serdes_Init](#)) in different modes of operation

3.3 Hardware Resources

The hardware configured by the Serdes driver is the same between derivatives.

Physical Serdes Channels:

S32G2XX: Serdes_0, Serdes_1

S32G3XX: Serdes_0, Serdes_1

3.4 Deviations from Requirements

N/A

3.5 Driver Limitations

- With internal clock the SerDes_Init has to be called after Mcu_Init.
- After SerDes is initialized, MUX for SerDes clocks provided to PFE/GMAC has to be changed again (Eg. by calling Mcu_Init again).
- After coming back to normal state from power down mode, Serdes can not return to Link Up state.

3.6 Driver usage and configuration tips

The s32 SoCs have two SerDeses. Each SerDes has two lanes and contains shared PHY with specific protocol wrapper above for PCIe and SGMII(XPCS). Phy contains two PLLs that have different usage for SGMII. This allows to run bifurcation of protocols or SerDes speeds on different lanes.

S32G2 mapping to XPCS:

SerDes	XPCS0	XPCS1
0	GMAC	PFE2
1	PFE0	PFE1

PHY PLLs usage for SGMII:

- PLLA - Slow PLL that should be connected to lane as reference for 1G operation
- PLLB - Fast PLL that should be connected to lane as reference for 2.5G operation

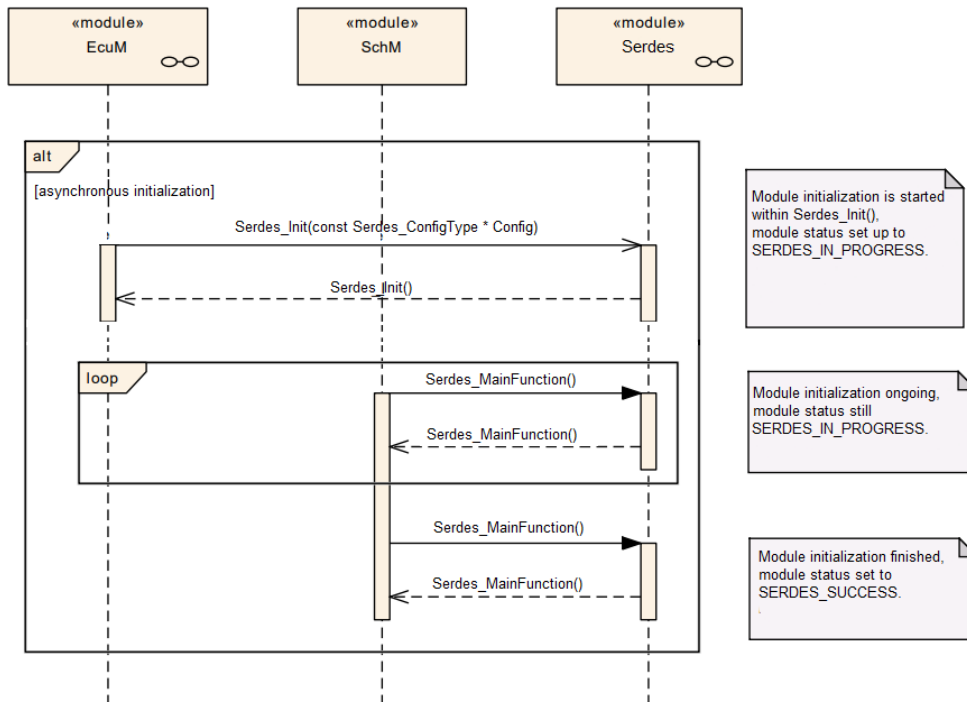
Modes supported (lane0/lane1):

- PCIe_X2 (PCIe(2x))
- PCIe_SGMII_XPCS0 (PCIe(1x)/SGMII(XPCS0) bifurcation)
- PCIe_SGMII_XPCS1 (PCIe(1x)/SGMII(XPCS1) bifurcation)
- SGMII(XPCS0)/SGMII(XPCS1) bifurcation
- SGMII(XPCS0)/SGMII(XPCS1) bifurcation

When auto-negotiation is used, SerdesTimeoutDuration should be chosen accordingly (usually greater than 65536), as auto-negotiation requires longer time to complete.

With external clock, SerdesTimeoutDuration must be at least 100000.

[Serdes_Init\(\)](#) function starts the initialization, to complete it, [Serdes_MainFunction\(\)](#) should be called multiple times, until the module is fully initialized.



3.7 Runtime errors

The driver generates the following DET errors at runtime.

Function	Error Code	Condition triggering the error
Serdes_CheckCurrentCoreId	SERDES_E_PARAM_CONFIG	Invalid resource configuration in current core.
Serdes_Init	SERDES_E_INIT_IN_PROGRESS	Invalid state.
Serdes_Init	SERDES_E_INVALID_POINTER	Invalid pointer parameter.
Serdes_Init	SERDES_E_PARAM_CONFIG	Invalid resource configuration in current core.
Serdes_GetJobResult	SERDES_E_UNINIT	The driver is in an uninitialized state.
Serdes_GetStatus	SERDES_E_UNINIT	The driver is in an uninitialized state.
Serdes_GetStatus	SERDES_E_INVALID_PARAMETER	Invalid input parameter.
Serdes_SetMode	SERDES_E_UNINIT	The driver is in an uninitialized state.
Serdes_SetMode	SERDES_E_INVALID_PARAMETER	Invalid input parameter.
Serdes_GetVersionInfo	SERDES_E_INVALID_POINTER	Invalid pointer parameter.

3.8 Symbolic Names Disclaimer

All containers having symbolicNameValue set to TRUE in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [Serdes](#)
 - Container [SerdesGeneral](#)
 - * Parameter [SerdesDevErrorDetect](#)
 - * Parameter [SerdesSetModeApi](#)
 - * Parameter [SerdesVersionInfoApi](#)
 - * Parameter [SerdesEnableUserModeSupport](#)
 - * Parameter [SerdesMulticoreSupport](#)
 - * Parameter [SerdesTimeoutMethod](#)
 - * Parameter [SerdesTimeoutDuration](#)
 - * Parameter [SerdesMainFunctionPeriod](#)
 - * Parameter [SerdesJobDone](#)
 - * Parameter [SerdesJobFailed](#)
 - * Reference [SerdesEcucPartitionRef](#)
 - Container [SerdesGlobalConfig](#)
 - * Container [SerdesChannel](#)
 - Parameter [SerdesChannelId](#)
 - Parameter [SerdesHwChannel](#)
 - Parameter [SerdesMode](#)
 - Parameter [SerdesPhyClock](#)
 - Parameter [SerdesClock](#)
 - Parameter [UseXpcs0](#)
 - Parameter [UseXpcs1](#)
 - Reference [SerdesChannelEcucPartitionRef](#)
 - Container [XPCS_0](#)
 - Parameter [XpcsSpeed0](#)
 - Parameter [XpcsDuplex0](#)
 - Parameter [XPCS0_AN](#)
 - Parameter [SerdesLoopback_0](#)
 - Container [XPCS_1](#)
 - Parameter [XpcsSpeed1](#)

- Parameter [XpcsDuplex1](#)
- Parameter [XPCS1_AN](#)
- Parameter [SerdesLoopback_1](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module Serdes

Configuration of the Serdes (SerdesDriver) module

Included containers:

- [SerdesGeneral](#)
- [SerdesGlobalConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	false
supportedConfigVariants	VARIANT-PRE-COMPILE

4.2 Container SerdesGeneral

Container for common configuration options

Included subcontainers:

Tresos Configuration Plug-in

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter SerdesDevErrorDetect

Switches the development error detection and notification on or off.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.4 Parameter SerdesSetModeApi

Pre-processor switch to enable and disable availability of the API Serdes_SetMode().

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.5 Parameter SerdesVersionInfoApi

Pre-processor switch to enable and disable availability of the API Serdes_GetVersionInfo().

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.6 Parameter SerdesEnableUserModeSupport

When this parameter is enabled, the Serdes module will adapt to run from User Mode, with the following measures:

(if applicable) a) configuring REG_PROT for the Serdes Controllers so that the registers under protection eth be accessed from user mode by setting UAA bit in REG_PROT_GCR to 1.

(if applicable) b) using 'call trusted function' stubs for all internal function calls that access registers requiring supervisor mode.

(if applicable) c) other module specific measures for more information, please see chapter 5.7 User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.7 Parameter SerdesMulticoreSupport

This parameter globally enables the possibility to support multicore. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note This is an Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.8 Parameter SerdesTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM', 'OSIF_COUNTER_DUMMY']

4.9 Parameter SerdesTimeoutDuration

The unit of measurement is given in number of microseconds.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	4294967295
min	1

4.10 Parameter SerdesMainFunctionPeriod

The period between successive calls to the Serdes_MainFunction() in seconds.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.001
max	65.535
min	0.001

4.11 Parameter SerdesJobDone

Serdes initialization job completed callback. This function will be called when initialization is completed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Serdes__JobDoneCallback

4.12 Parameter SerdesJobFailed

Serdes initialization job failed callback. This function will be called when initialization had failed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	Serdes__JobFailedCallback

4.13 Reference SerdesEcucPartitionRef

Maps the SERDES driver to zero or multiple ECUC partitions to make the driver API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.14 Container SerdesGlobalConfig

This container contains the global configuration parameter of the Serdes driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exist once per configuration set.

Included subcontainers:

- [SerdesChannel](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.15 Container SerdesChannel

This container contains the configuration (parameters) of the Serdes Controller(s).

Note: "User should use unique names for naming the Serdes channels across different SerdesGlobalConfig Sets."

Included subcontainers:

- [XPCS_0](#)
- [XPCS_1](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

4.16 Parameter SerdesChannelId

Identifies the Serdes channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	1
min	0

4.17 Parameter SerdesHwChannel

Selects the physical Serdes Channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	Serdes_0
literals	['Serdes_0', 'Serdes_1']

4.18 Parameter SerdesMode

Selects the mode for Serdes.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SGMII_MODE_3
literals	['PCIE_X2', 'PCIE_SGMII_XPCS_0', 'PCIE_SGMII_XPCS_1', 'SGMII_MODE_3', 'SGMII_MODE_4']

4.19 Parameter SerdesPhyClock

Selects the PHY clock, internal or external.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SERDES_PHY_CLK_INT
literals	['SERDES_PHY_CLK_INT', 'SERDES_PHY_CLK_EXT']

4.20 Parameter SerdesClock

Selects the Serdes clock, 100MHz or 125 Mhz.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	SERDES_CLK_100MHZ
literals	['SERDES_CLK_100MHZ', 'SERDES_CLK_125MHZ']

4.21 Parameter UseXpcs0

XPCS0 switch

Use XPCS0 in case GMAC or PFE0 is used.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.22 Parameter UseXpcs1

XPCS1 switch

Use XPCS1 in case PFE1 or PFE2 is used.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.23 Reference SerdesChannelEcucPartitionRef

Maps the Serdes Channel to zero a multiple ECUC partitions. The ECUC partitions referenced are a subset of the ECUC partitions where the related SerdesChannelEcucPartitionRef is mapped to.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.24 Container XPCS_0

This container contains the configuration (parameters) of the Serdes Controller(s).

Note: "User should use unique names for naming the Serdes channels across different SerdesGlobalConfig Sets."

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.25 Parameter XpcsSpeed0

Selects the PCS speed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SERDES_MAC_SPEED_10_MBPS
literals	['SERDES_MAC_SPEED_10_MBPS', 'SERDES_MAC_SPEED_100_↵ MBPS', 'SERDES_MAC_SPEED_1000_MBPS', 'SERDES_MAC_SPEED_↵ _2500_MBPS']

4.26 Parameter XpcsDuplex0

Selects the PCS speed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SERDES_MAC_DUPLEX_FULL
literals	['SERDES_MAC_DUPLEX_HALF', 'SERDES_MAC_DUPLEX_FULL']

4.27 Parameter XPCS0_AN

XPCS autonegotiation

Switches the XPCS autonegotiation function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.28 Parameter SerdesLoopback_0

SerdesLoopback

Switches the PMA loopback function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.29 Container XPCS_1

This container contains the configuration (parameters) of the Serdes Controller(s).

Note: "User should use unique names for naming the Serdes channels across different SerdesGlobalConfig Sets."

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.30 Parameter XpcsSpeed1

Selects the PCS speed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SERDES_MAC_SPEED_10_MBPS
literals	['SERDES_MAC_SPEED_10_MBPS', 'SERDES_MAC_SPEED_100_↵ MBPS', 'SERDES_MAC_SPEED_1000_MBPS', 'SERDES_MAC_SPEED_↵ _2500_MBPS']

4.31 Parameter XpcsDuplex1

Selects the PCS speed.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	SERDES_MAC_DUPLEX_FULL
literals	['SERDES_MAC_DUPLEX_HALF', 'SERDES_MAC_DUPLEX_FULL']

4.32 Parameter XPCS1__AN

XPCS autonegotiation

Switches the XPCS autonegotiation function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.33 Parameter SerdesLoopback_1

SerdesLoopback

Switches the PMA loopback function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.34 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.35 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.36 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4

Property	Value
max	4
min	4

4.37 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.38 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.39 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
default Value	4
max	4
min	4

4.40 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
default Value	0
max	0
min	0

4.41 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.42 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>__>VendorId>__<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name

Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.43 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

This chapter describes the Tresos configuration plug-in for the Serdes Driver. The most of the parameters are described below.



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

Serdes HLD	34
Serdes IPL	40

Chapter 6

Module Documentation

6.1 Serdes HLD

6.1.1 Detailed Description

Data Structures

- struct [Serdes_HwUnitConfigType](#)
Structure that contains Serdes Hw configuration. [More...](#)
- struct [Serdes_ConfigType](#)
This type contains initialization data. [More...](#)

Macros

- `#define SERDES\_E\_INVALID\_POINTER`
API service called with invalid configuration pointer.
- `#define SERDES\_E\_INIT\_IN\_PROGRESS`
API service called with invalid configuration pointer.
- `#define SERDES\_E\_UNINIT`
API service called with invalid configuration pointer.
- `#define SERDES\_E\_INVALID\_PARAMETER`
API service called with invalid configuration pointer.
- `#define SERDES\_E\_PARAM\_CONFIG`
The SERDES module is not properly configured.

Types Reference

- typedef uint8 [Serdes_HwUnitType](#)
This gives the numeric ID (hardware number) of a Serdes hw Unit.

Enum Reference

- enum [Serdes__StatusType](#)
Definition for Serdes status.
- enum [Serdes__JobResultType](#)
Definition for job result.

Function Reference

- void [Serdes__Init](#) (const [Serdes__ConfigType](#) *Config)
Starts initialization of serdes module.
- [Serdes__JobResultType](#) [Serdes__GetJobResult](#) (void)
Returns job result.
- [Serdes__StatusType](#) [Serdes__GetStatus](#) (uint8 Channel)
Returns driver status.
- void [Serdes__SetMode](#) (uint8 Channel, [Serdes__ModeType](#) Mode)
Sets the power mode for the selected channel.
- void [Serdes__GetVersionInfo](#) (Std__VersionInfoType *VersionInfo)
Gets version information.

6.1.2 Data Structure Documentation

6.1.2.1 struct Serdes__HwUnitConfigType

Structure that contains Serdes Hw configuration.

It contains the information specific to one Serdes Hw unit

Definition at line 134 of file [Serdes__Types.h](#).

Data Fields

Type	Name	Description
const Serdes__HwUnitType	Serdes__HwUnit	Numeric instance value of Serdes Hw Unit. < Structure containing the hardware specific configuration for the channel.
const Serdes__Ipw__HwChannelConfigType *	HwChannelConfig	

6.1.2.2 struct Serdes__ConfigType

This type contains initialization data.

This contains initialization data for the Serdes driver. It shall contain:

- The number of Serdes modules to be configured
- Serdes dependent properties for used HW units

Definition at line 151 of file [Serdes_Types.h](#).

Data Fields

- uint32 [CoreId](#)
Configuration CoreID.

6.1.2.2.1 Field Documentation

6.1.2.2.1.1 CoreId uint32 CoreId

Configuration CoreID.

Pointer to Serdes hardware unit configuration

Definition at line 154 of file [Serdes_Types.h](#).

6.1.3 Macro Definition Documentation

6.1.3.1 SERDES_E_INVALID_POINTER

```
#define SERDES_E_INVALID_POINTER
```

API service called with invalid configuration pointer.

The Serdes Driver module shall report the development error "SERDES_E_INVALID_POINTER (0x01)", when API Service is called with invalid configuration pointer.

Definition at line 149 of file [Serdes.h](#).

6.1.3.2 SERDES_E_INIT_IN_PROGRESS

```
#define SERDES_E_INIT_IN_PROGRESS
```

API service called with invalid configuration pointer.

The Serdes Driver module shall report the development error "SERDES_E_INIT_IN_POINTER (0x02)", when API Service is called and Serdes initialization is already in progress.

Definition at line 159 of file [Serdes.h](#).

6.1.3.3 SERDES_E_UNINIT

```
#define SERDES_E_UNINIT
```

API service called with invalid configuration pointer.

The Serdes Driver module shall report the development error "SERDES_E_UNINIT (0x03)", when API Service is called and Serdes initialization is not started.

Definition at line 169 of file [Serdes.h](#).

6.1.3.4 SERDES_E_INVALID_PARAMETER

```
#define SERDES_E_INVALID_PARAMETER
```

API service called with invalid configuration pointer.

The Serdes Driver module shall report the development error "SERDES_E_INVALID_PARAMETER (0x04)", when API Service is called with out of range parameter.

Definition at line 179 of file [Serdes.h](#).

6.1.3.5 SERDES_E_PARAM_CONFIG

```
#define SERDES_E_PARAM_CONFIG
```

The SERDES module is not properly configured.

Definition at line 206 of file [Serdes.h](#).

6.1.4 Types Reference

6.1.4.1 Serdes_HwUnitType

```
typedef uint8 Serdes_HwUnitType
```

This gives the numeric ID (hardware number) of a Serdes hw Unit.

Definition at line 128 of file [Serdes_Types.h](#).

6.1.5 Enum Reference

6.1.5.1 Serdes_StatusType

```
enum Serdes_StatusType
```

Definition for Serdes status.

Enumerator

SERDES_UNINIT	Serdes was not initialized.
SERDES_SUCCESS	Serdes initialization was successful.
SERDES_INIT_IN_PROGRESS	Serdes initialization is in progress.
SERDES_ERROR	Serdes initialization was not successful.

Definition at line 102 of file [Serdes_Types.h](#).

6.1.5.2 Serdes_JobResultType

enum [Serdes_JobResultType](#)

Definition for job result.

Enumerator

SERDES_INIT_OK	Init job is completed and successful.
SERDES_INIT_PENDING	Init job is in progress.
SERDES_INIT_FAILED	Init job has failed.

Definition at line 115 of file [Serdes_Types.h](#).

6.1.6 Function Reference

6.1.6.1 Serdes_Init()

```
void Serdes_Init (
    const Serdes\_ConfigType * Config )
```

Starts initialization of serdes module.

Initialization is done asynchronous and will not be completed after calling this function. [Serdes_MainFunction\(\)](#) should be called multiple times until initialization is completed.

Parameters

in	<i>Config</i>	Serdes configuration structure.
----	---------------	---------------------------------

Returns

void

6.1.6.2 Serdes__GetJobResult()

```
Serdes_JobResultType Serdes_GetJobResult (
    void )
```

Returns job result.

This function returns the initialization job result. In case the initialization is completed it will return SERDES_↵INIT_OK, otherwise it will return SERDES_INIT_PENDING if initialization is still in progress or SERDES_↵INIT_FAILED if initialization failed to complete.

Returns

void

6.1.6.3 Serdes__GetStatus()

```
Serdes_StatusType Serdes_GetStatus (
    uint8 Channel )
```

Returns driver status.

Returns

void

6.1.6.4 Serdes__SetMode()

```
void Serdes_SetMode (
    uint8 Channel,
    Serdes_ModeType Mode )
```

Sets the power mode for the selected channel.

Returns

void

6.1.6.5 Serdes_GetVersionInfo()

```
void Serdes_GetVersionInfo (
    Std_VersionInfoType * VersionInfo )
```

Gets version information.

Returns

void

6.2 Serdes IPL

6.2.1 Detailed Description

Data Structures

- struct [Serdes_Ip_XpcsConfigType](#)
Definition for Serdes XPCS config. [More...](#)
- struct [Serdes_Ip_ConfigType](#)
This type contains initialization data. [More...](#)

Function Reference

- Serdes_Ip_StatusType [Serdes_Ip_Init](#) (uint8 Instance, const [Serdes_Ip_ConfigType](#) *Serdes)
Starts initialization of serdes module.
- Serdes_Ip_JobResultType [Serdes_Ip_GetJobResult](#) (uint8 Instance)
Gets job result.
- Serdes_Ip_StatusType [Serdes_Ip_GetStatus](#) (uint8 Instance)
Returns serdes status.
- Serdes_Ip_StatusType [Serdes_Ip_MainFunction](#) (uint8 Instance)
Continues initialization of serdes module.
- void [Serdes_Ip_SetMode](#) (uint8 Instance, Serdes_Ip_XpcsModeType XpcsMode)
Set the power mode of the XPCS.

6.2.2 Data Structure Documentation

6.2.2.1 struct Serdes_Ip_XpcsConfigType

Definition for Serdes XPCS config.

Definition at line 150 of file [Serdes_Ip_Types.h](#).

Data Fields

- boolean [LoopbackEnabled](#)
Loopback of PMA enabled.
- Serdes_Ip_MacSpeedType [XpcsSpeed](#)
XPCS speed.
- Serdes_Ip_MacDuplexType [XpcsDuplex](#)
XPCS duplex.
- boolean [XpcsAN](#)
XPCS autonegotiation.

6.2.2.1.1 Field Documentation

6.2.2.1.1.1 LoopbackEnabled `boolean LoopbackEnabled`

Loopback of PMA enabled.

Definition at line 152 of file [Serdes_Ip_Types.h](#).

6.2.2.1.1.2 XpcsSpeed `Serdes_Ip_MacSpeedType XpcsSpeed`

XPCS speed.

Definition at line 153 of file [Serdes_Ip_Types.h](#).

6.2.2.1.1.3 XpcsDuplex `Serdes_Ip_MacDuplexType XpcsDuplex`

XPCS duplex.

Definition at line 154 of file [Serdes_Ip_Types.h](#).

6.2.2.1.1.4 XpcsAN `boolean XpcsAN`

XPCS autonegotiation.

Definition at line 155 of file [Serdes_Ip_Types.h](#).

6.2.2.2 struct Serdes_Ip_ConfigType

This type contains initialization data.

This contains initialization data for the Serdes driver.

Definition at line 163 of file [Serdes_Ip_Types.h](#).

Data Fields

- uint32 [PhyReferenceClock](#)
Internal or external clock.
- Serdes_Ip_ClockType [SerdesRefClock](#)
100 MHz or 125 MHz
- Serdes_Ip_ModeType [SerdesMode](#)
Functional mode of Serdes (eg. PCIE_X2, SGMII, SERDES_PCIE_XPCS_0)
- const [Serdes_Ip_XpcsConfigType](#) * [XPCS0](#)
Configuration for XPCS0.
- const [Serdes_Ip_XpcsConfigType](#) * [XPCS1](#)
Configuration for XPCS1.

6.2.2.2.1 Field Documentation

6.2.2.2.1.1 [PhyReferenceClock](#) uint32 [PhyReferenceClock](#)

Internal or external clock.

Definition at line 165 of file [Serdes_Ip_Types.h](#).

6.2.2.2.1.2 [SerdesRefClock](#) Serdes_Ip_ClockType [SerdesRefClock](#)

100 MHz or 125 MHz

Definition at line 166 of file [Serdes_Ip_Types.h](#).

6.2.2.2.1.3 [SerdesMode](#) Serdes_Ip_ModeType [SerdesMode](#)

Functional mode of Serdes (eg. PCIE_X2, SGMII, SERDES_PCIE_XPCS_0)

Definition at line 167 of file [Serdes_Ip_Types.h](#).

6.2.2.2.1.4 XPCS0 `const Serdes_Ip_XpcsConfigType* XPCS0`

Configuration for XPCS0.

Definition at line 168 of file [Serdes_Ip_Types.h](#).

6.2.2.2.1.5 XPCS1 `const Serdes_Ip_XpcsConfigType* XPCS1`

Configuration for XPCS1.

Definition at line 169 of file [Serdes_Ip_Types.h](#).

6.2.3 Function Reference**6.2.3.1 Serdes_Ip_Init()**

```
Serdes_Ip_StatusType Serdes_Ip_Init (
    uint8 Instance,
    const Serdes_Ip_ConfigType * Serdes )
```

Starts initialization of serdes module.

This function initializes PLLs, electrical parameters and xpcs logic.

6.2.3.2 Serdes_Ip_GetJobResult()

```
Serdes_Ip_JobResultType Serdes_Ip_GetJobResult (
    uint8 Instance )
```

Gets job result.

This function returns the status of the current job.

6.2.3.3 Serdes_Ip_GetStatus()

```
Serdes_Ip_StatusType Serdes_Ip_GetStatus (
    uint8 Instance )
```

Returns serdes status.

This function returns the current driver state.

6.2.3.4 Serdes_Ip_MainFunction()

```
Serdes_Ip_StatusType Serdes_Ip_MainFunction (
    uint8 Instance )
```

Continues initialization of serdes module.

This function will complete the initialization of serdes module.

6.2.3.5 Serdes_Ip_SetMode()

```
void Serdes_Ip_SetMode (
    uint8 Instance,
    Serdes_Ip_XpcsModeType XpcsMode )
```

Set the power mode of the XPCS.

This function changes and waits for the XPCS to achieve the power mode requested

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

